

SAVEETHA UNIVERSITY

Department of Computer Science and Engineering

COMMON COURSE ASSIGNMENT REPORT

Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System using Java

Course Code & Name: CSA09 — Programming in Java (Slot B)

Submitted by (Team):

1. _____Anish Ebenezer_____ (Reg. No: 192511337)— Module 1 Lead
2. _____Pushparaj B_____ (Reg. No: 192421136) — Module 2 Lead
3. _____N.NESARAJAN_____ (Reg. No: 192511386)— Module 3 Lead
4. _____Asif ali khan_____ (Reg. No: 192521009) — Module 4 Lead
5. _____G Sreekanth_____ (Reg. No:192311422)— Module 5 Lead

Faculty: Dr. Thalpathi Rajasekaran R

A. Assignment Information

Department	Computer Science and Engineering
Programme	Computer Science and Engineering
Course Code & Course Name	CSA09 — Programming in Java (Slot B)
Academic Year / Batch	2023-2027
Faculty Name	Dr. Thalpathi Rajasekaran
Assignment Title	Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System using Java
Maximum Marks	100
Course Outcome(s) — CO	CO1: OOP principles (classes, encapsulation, inheritance, polymorphism). CO2: Collection Framework (List, ArrayList, Set, Map, HashMap, Hashtable) with Iterator/ListIterator and generics. CO3: Multithreading (priorities, synchronization, inter-thread communication) and exception handling.
Bloom's Taxonomy Level	L2 — Understand; L3 — Apply; L4 — Analyse
SDG Mapping (SDG 1–17)	SDG 3 — Good Health and Well-being; SDG 9 — Industry, Innovation and Infrastructure; SDG 10 — Reduced Inequalities
Industry / Societal Relevance	Digitises hospital front-office and pharmacy workflows that are still largely paper-based in many small and mid-sized clinics, reducing appointment conflicts, medicine stock-outs and missed follow-ups.

Table of Contents

A. Assignment Information.....	2
Table of Contents	2
1. Problem Statement and Problem Formulation	5
1.1 Problem Statement	5
1.2 Problem Formulation.....	5
2. Objective and Expected Outcomes	5
2.1 Objectives	5
2.2 Expected Outcomes.....	6
3. Requirements, Constraints and Assumptions	6
3.1 Functional Requirements	6
3.2 Non-Functional / Technical Constraints	7
3.3 Assumptions	7
4. Application of Relevant Course Knowledge / Concepts	8
4.1 CO1 — OOP: Classes, Encapsulation, Inheritance, Polymorphism.....	8
4.2 CO2 — Collection Framework, Generics, Iterators	8

4.3 CO3 — Exception Handling, Multithreading, Synchronization.....	9
5. Design / Proposed Solution / Methodology	11
5.1 Architectural Overview.....	11
5.2 Class Design Summary.....	11
5.3 Key Design Decisions and Alternatives Considered.....	12
6. Algorithm / Pseudocode / Flowchart	13
6.1 Flowchart — Book Appointment (described).....	13
7. Implementation / Source Code and Environment / Tools Used	14
7.1 Development Environment	14
7.2 Representative Source Code	14
7.2.1 Module 1 — Polymorphic factory (Medicine)	14
7.2.2 Module 2 — Collections-backed booking (HospitalService)	14
7.2.3 Module 3 — Exception-safe menu dispatch (ConsoleApp).....	14
7.2.4 Module 4 — Thread-safe stock reduction (Inventory + JDBC)	15
7.2.5 Module 5 — Menu loop and Applet action dispatch	15
8. Test Cases and Expected/Actual Results	16
9. Execution Screenshots / Outputs	17
10. Results and Validation	18
10.1 Functional Coverage	18
10.2 Observations.....	19
11. Analysis, Comparison, Trade-offs and Justification of the Final Solution	20
11.1 Design Alternatives Considered	20
11.2 Trade-offs of the Final Solution	20
11.3 Justification.....	21
12. Broader Considerations / SDG Relevance	22
12.1 SDG 3 — Good Health and Well-being.....	22
12.2 SDG 9 — Industry, Innovation and Infrastructure.....	22
12.3 SDG 10 — Reduced Inequalities	22
12.4 Ethics, Safety and Professional Responsibility	22
13. Conclusion, Limitations and Possible Improvements	22
13.1 Conclusion	22
13.2 Limitations	23
13.3 Possible Improvements	23
14. Individual Contribution of Group Members.....	24
Member 1 — Core entity classes and inheritance hierarchy	24
Member 2 — The central service facade and its collections.....	24
Member 3 — Custom exceptions and the notification class hierarchy	24

Member 4 — Concurrency primitives and the database access layer	25
Member 5 — Console and Applet front ends, entry point, overall test coordination	25
15. References	27
16. One-Page Individual Reflection	28
16.1 Individual Reflection — Member 1	28
Design/Development Decisions.....	Error! Bookmark not defined.
Challenges Faced	Error! Bookmark not defined.
Learning Gained.....	Error! Bookmark not defined.
SDG Connection.....	Error! Bookmark not defined.
Course Outcome Attainment	Error! Bookmark not defined.
16.2 Individual Reflection — Member 2	Error! Bookmark not defined.
Design/Development Decisions.....	Error! Bookmark not defined.
Challenges Faced	Error! Bookmark not defined.
Learning Gained.....	Error! Bookmark not defined.
SDG Connection.....	Error! Bookmark not defined.
Course Outcome Attainment	Error! Bookmark not defined.
16.3 Individual Reflection — Member 3	Error! Bookmark not defined.
Design/Development Decisions.....	Error! Bookmark not defined.
Challenges Faced	Error! Bookmark not defined.
Learning Gained.....	Error! Bookmark not defined.
SDG Connection.....	Error! Bookmark not defined.
Course Outcome Attainment	Error! Bookmark not defined.
16.4 Individual Reflection — Member 4	Error! Bookmark not defined.
Design/Development Decisions.....	Error! Bookmark not defined.
Challenges Faced	Error! Bookmark not defined.
Learning Gained.....	Error! Bookmark not defined.
SDG Connection.....	Error! Bookmark not defined.
Course Outcome Attainment	Error! Bookmark not defined.
16.5 Individual Reflection — Member 5	Error! Bookmark not defined.
Design/Development Decisions.....	Error! Bookmark not defined.
Challenges Faced	Error! Bookmark not defined.
Learning Gained.....	Error! Bookmark not defined.
SDG Connection.....	Error! Bookmark not defined.
Course Outcome Attainment	Error! Bookmark not defined.

1. Problem Statement and Problem Formulation

1.1 Problem Statement

Hospitals and clinics that rely on manual registers for patient appointments and pharmacy stock face recurring, avoidable problems: doctors get double-booked or under-booked because free slots are not visible at a glance; patients with genuine follow-up needs are turned away when a doctor is full instead of being queued; medicine stock is dispensed without a reliable check against what is actually on the shelf, leading to stock-outs discovered only when a patient is standing at the counter; and nobody is reminded of an upcoming appointment or a falling stock level until it is too late. The assignment requires the design, implementation and evaluation of a menu-driven Java application — the Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System — that automates this workflow end-to-end: registering patients and doctors, adding and tracking medicines, displaying available slots and stock, booking and cancelling appointments with a waitlist fallback, dispensing medicine with stock validation, generating patient-visit and pharmacy-utilization reports, and dispatching appointment-reminder and low-stock notifications, all backed by persistent JDBC storage.

1.2 Problem Formulation

- Core entities to model: Patient (with sub-categories), Doctor, Appointment, Medicine (with sub-types), Inventory and Notification.
- Core workflow to automate: register → discover availability → book/cancel → dispense → report → notify, each step guarded by validation and exception handling.
- Concurrency to simulate: two patients competing for the same doctor slot, and a background dispatcher delivering queued notifications, without corrupting shared appointment or stock data.
- Persistence to provide: every create/update operation must survive a restart, via JDBC against an embedded database (hospital.db).
- Interfaces to provide: a console menu (primary) and a minimal AWT Applet GUI (secondary, to demonstrate an alternative front-end over the same service layer).

The problem was decomposed into five cooperating modules — domain model, collections/business logic, exceptions & notifications, concurrency & persistence, and presentation/integration — described in detail in Section 5 and assigned one per team member in Section 14.

2. Objective and Expected Outcomes

2.1 Objectives

1. Design a set of encapsulated, cohesive domain classes (Patient, Doctor, Medicine, Appointment, Notification) using constructors, private fields and public accessor/mutator methods.

2. Apply inheritance and run-time polymorphism so that patient categories (Regular / Senior Citizen / Pediatric) automatically get the correct consultation-fee waiver, medicine types (Tablet / Syrup) return the correct dosage instructions, and notification types (reminder / low-stock) format their own message — all through a single supertype reference.
3. Use the Java Collection Framework — ArrayList, HashMap, HashSet, Hashtable, LinkedList — together with generics and Iterator / ListIterator to store, search, update, traverse and remove records without exposing internal storage.
4. Implement a layered exception-handling strategy using custom checked exceptions (InvalidPatientIdException, DuplicateAppointmentException, DoctorFullyBookedException, InvalidInputException, OutOfStockException) so that no invalid operation ever terminates the program.
5. Build multithreaded behaviour: a background notification dispatcher using wait/notify inter-thread communication, and concurrent appointment-booking threads with explicit thread priorities contending for the same synchronized resource.
6. Persist all entities through JDBC (DriverManager, PreparedStatement, ResultSet) so that data survives across runs.
7. Provide a menu-driven console interface (and a demonstrative Applet GUI) that exercises every one of the above features and produces patient-visit and pharmacy-utilization reports.

2.2 Expected Outcomes

- A compiling, runnable Java program (17-option console menu + optional Applet) that registers, books, cancels, dispenses, searches, updates and reports correctly against the seeded demo data (D001–D003, M001–M003, P001–P003).
- Demonstrable evidence — through code and test cases — of every CO1/CO2/CO3 concept listed in the rubric: encapsulation, inheritance, polymorphism, ArrayList/Set/HashMap/Hashtable with generics and Iterator/ListIterator, custom exceptions, thread priorities, synchronization and inter-thread communication.
- A waitlist mechanism that automatically promotes the next patient when a booked slot is cancelled, and a low-stock alert that fires the moment stock drops to or below the reorder level.
- A short, reproducible test log (Section 8) mapping each menu option to an expected and an observed result.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

- Register a patient (name, age, phone, category) with age in the range 1–120 and a phone number of at least 8 digits; the system must assign a sequential patient ID (P001, P002, ...).
- Register a doctor (name, specialization, 1–8 default slots drawn from a fixed catalog of eight clinic time-slots) and a medicine (name, TABLET/SYRUP type, stock, reorder level, unit price).

- Display, on demand, every doctor's free slots and current waitlist size, and every medicine's stock and dosage instructions.
- Book an appointment against a preferred or first-available slot; reject a duplicate active booking for the same patient/doctor pair; if no slot is free, enqueue the patient on that doctor's waitlist instead of failing silently.
- Cancel a booked appointment, free its slot, and automatically promote the next waitlisted patient into that slot if one exists.
- Search a patient by ID; update a patient's name/phone/age or a medicine's name/stock, re-persisting the change via JDBC.
- Dispense a quantity of medicine only if sufficient stock exists; reduce stock atomically and raise a low-stock alert once stock falls to or below the reorder level.
- Run a concurrent-booking demonstration: two MAX_PRIORITY threads attempt to book the same doctor/slot simultaneously so the collision is handled safely.
- Generate a patient-visit report (booked/cancelled counts, fee collection) and a pharmacy-utilization report (units on shelf, low-stock count, delivered notifications).

3.2 Non-Functional / Technical Constraints

- Language/runtime: Java (JDK 8+ syntax observed in the codebase; the Applet path notes `java.applet.Applet` is unavailable from JDK 9 onward, so a project-local `smarthospital.applet.Applet` substitute is used).
- Persistence: JDBC only (`java.sql` — `DriverManager`, `Connection`, `PreparedStatement`, `ResultSet`) against a local embedded store, `hospital.db`; no external RDBMS server dependency.
- Concurrency safety: all mutations to shared state (appointment slots, waitlists, inventory counts) must occur inside synchronized methods/blocks; the notification queue must use `wait/notify`, not busy-polling.
- Usability: a single, self-explanatory numbered console menu (0–17) must remain the primary interface; the Applet is an optional secondary GUI over the same `HospitalService`.
- Portability: build/run via simple scripts (`compile.bat` / `run.bat`) with no external build tool required.

3.3 Assumptions

- A single clinic location and a single day's slot catalog (09:00–17:00, eight fixed slots) are sufficient for the assignment's scope; multi-day scheduling is out of scope.
- Demo/seed data (3 doctors, 3 medicines, 3 patients) is acceptable for first-run testing in the absence of a real hospital data feed.
- Notification "delivery" means the message is dequeued, formatted and recorded (and shown in reports); actual SMS/e-mail gateways are outside the assignment's scope.
- Only one JVM instance accesses `hospital.db` at a time — the assignment does not need to handle multi-process database contention.

4. Application of Relevant Course Knowledge / Concepts

The table below maps each Course Outcome to the specific language feature used and the exact class/method in the codebase where it is demonstrated, so that the connection between taught theory and implemented code is explicit and auditable.

4.1 CO1 — OOP: Classes, Encapsulation, Inheritance, Polymorphism

Concept	Where applied	Explanation
Encapsulation / data hiding	Patient, Doctor, Medicine, Appointment (model package)	All fields are private; access is only through getters/setters or constructors, e.g. Patient.getId(), Medicine.setStock().
Abstract class + abstract methods	Patient, Medicine, HospitalNotification	Each declares abstract methods (getCategory()/getConsultationFee(), getType()/getDosageInstructions(), getType()/formatMessage()) that only make sense per concrete subtype.
Inheritance	RegularPatient / SeniorCitizenPatient / PediatricPatient extend Patient; TabletMedicine / SyrupMedicine extend Medicine; AppointmentReminderNotification / LowStockNotification extend HospitalNotification	Shared state and behaviour live once in the supertype; type-specific behaviour is overridden in each subclass.
Run-time polymorphism	Patient.create(...) / Medicine.create(...) factory methods; ConsoleApp calling p.getConsultationFee(...)	The caller holds a Patient or Medicine reference but the JVM dispatches to the correct overridden method for the actual sub-type at run time — e.g. a senior citizen's fee waiver applies automatically without an if/else on category in the calling code.
Constructors & object creation	Doctor(id,name,spec,slots) auto-generates its default-slot Set	Constructors enforce that an object is never left in a partially-initialised state.

4.2 CO2 — Collection Framework, Generics, Iterators

Collection / API	Where applied	Purpose
ArrayList<Patient>/<Doctor>/<Medicine>/<Appointment>	HospitalService fields patients, doctors, medicines, appointments	Ordered, growable storage of every registered entity, returned to callers as defensive copies (new ArrayList<>(...)).
HashMap<String,X>	patientsById, doctorsById, medicinesById	O(1) average lookup by business ID for

Collection / API	Where applied	Purpose
		search/update/booking validation.
HashSet<String>	bookedSlotsByDoctor (per doctor), specializations, Doctor.defaultSlots (as LinkedHashSet)	Fast membership tests for "is this slot already booked" and unique specialization tracking.
LinkedList<String> (as a Queue)	waitlistByDoctor	FIFO waitlist per doctor; removeFirst() promotes the longest-waiting patient on cancellation.
Hashtable<String,Integer>	Inventory.stockByMedicineId	Thread-safe (legacy synchronized) collection explicitly required by the assignment for pharmacy stock, doubling as a second, cross-checked source of truth against Medicine.stock.
Iterator<T>	bookAppointment() duplicate scan, availableSlotsReport(), stockReport(), patientVisitReport(), pharmacyUtilizationReport()	Safe forward traversal for search/reporting without exposing the backing list.
ListIterator<Appointment>	cancelAppointment()	Bidirectional traversal capability used to locate the appointment to cancel by ID.
Generics	Throughout (ArrayList<Patient>, HashMap<String,Doctor>, etc.)	Compile-time type safety; removes the need for casts when retrieving elements.

4.3 CO3 — Exception Handling, Multithreading, Synchronization

Concept	Where applied	Purpose
Custom checked exceptions	InvalidPatientIdException, DuplicateAppointmentException, DoctorFullyBookedException, InvalidInputException, OutOfStockException (exception package)	Represent business-rule violations distinctly from programming errors, forcing callers to handle them explicitly.
try / catch / finally (multi-catch)	ConsoleApp.run(), HospitalApplet.actionPerformed()	A single multi-catch block (InvalidPatientIdException DuplicateAppointmentException DoctorFullyBookedException InvalidInputException

Concept	Where applied	Purpose
		OutOfStockException) reports business errors without crashing the menu loop; finally prints a spacer line each iteration.
Thread creation & priority	AppointmentBookingTask (extends Thread), started with Thread.MAX_PRIORITY in ConsoleApp.concurrentDemo()	Simulates two patients racing to book the same doctor/slot; priority influences scheduling preference.
synchronized methods/blocks	HospitalService.bookAppointment/cancelAppointment/dispenseMedicine (synchronized methods); synchronized(inventory) block in dispenseMedicine; Inventory's own synchronized put/getQuantity/reduce/keys	Guarantees only one thread mutates a doctor's booked-slot set or the shared stock count at a time, preventing race conditions and double-booking.
Inter-thread communication (wait/notify)	NotificationMonitor + NotificationDispatcher (concurrent package)	The dispatcher thread blocks (wait()) on an empty notification queue and is woken (notify()) the instant bookAppointment()/dispenseMedicine() enqueues a reminder or low-stock alert — avoiding a busy-wait poll loop.

5. Design / Proposed Solution / Methodology

5.1 Architectural Overview

The application follows a layered architecture inside a single smarthospital package tree, keeping each concern in its own sub-package so the five-way team split (Section 14) maps directly onto the source layout:

```

smarthospital
├── model/      Patient(+3 subtypes), Doctor, Medicine(+2 subtypes),
                Appointment, HospitalNotification(+2 subtypes)  -> Module 1 & part of 3
├── service/    HospitalService (singleton facade: collections, business rules) -> Module 2
├── exception/  InvalidPatientIdException, DuplicateAppointmentException,
                DoctorFullyBookedException, InvalidInputException,
                OutOfStockException                                -> Module 3
├── concurrent/ NotificationDispatcher, NotificationMonitor,
                AppointmentBookingTask                            -> Module 4
├── dao/         DatabaseManager (JDBC access layer)                -> Module 4
├── jdbc/        EmbeddedHospitalDriver                            -> Module 4
├── ui/          ConsoleApp, HospitalApplet, AppletViewer           -> Module 5
├── applet/      Applet (JDK9+ compatible base class)             -> Module 5
└── HospitalMain entry point (console by default, --applet flag) -> Module 5

```

HospitalService is implemented as a lazily-initialised singleton (getInstance()) that owns every in-memory collection and is the only class allowed to mutate them; ConsoleApp and HospitalApplet are thin presentation layers that call into it and translate results/exceptions into console or AWT output. This keeps business logic in exactly one place and independently testable from either front end.

5.2 Class Design Summary

Class	Responsibility	Key relationships
Patient (abstract) → RegularPatient / SeniorCitizenPatient / PediatricPatient	Identity + consultation-fee policy per category	Created via Patient.create(id,name,age,phone,category) factory
Doctor	Identity, specialization, fixed default time-slot Set	Referenced by Appointment.doctorId; slots tracked in HospitalService.bookedSlotsByDoctor
Medicine (abstract) → TabletMedicine / SyrupMedicine	Identity, stock, reorder level, price, dosage policy	Created via Medicine.create(...); mirrored in Inventory Hashtable
Appointment	Links a patient, a doctor, a slot and a status (BOOKED/CANCELLED)	Owned/mutated only by HospitalService
HospitalNotification (abstract) → AppointmentReminderNotification / LowStockNotification	Represents a queued alert with its own message format	Produced by HospitalService, queued in NotificationMonitor, consumed by NotificationDispatcher
Inventory	Thread-safe Hashtable- backed stock ledger	Used by HospitalService as the authoritative concurrent stock store
HospitalService	Facade + business rules + all collections + waitlists	Singleton; used by ConsoleApp and HospitalApplet

Class	Responsibility	Key relationships
DatabaseManager	JDBC CRUD for all four entity tables	Backs HospitalService.loadFromDatabase()/insert*/update*

5.3 Key Design Decisions and Alternatives Considered

- Factory methods (Patient.create, Medicine.create) over exposing constructors of every subclass directly — keeps ID-generation and category/type parsing centralized and lets ConsoleApp/HospitalApplet stay ignorant of concrete subclasses, which is a cleaner application of polymorphism than a switch in the UI layer.
- Hashtable chosen for Inventory specifically (rather than a synchronized HashMap wrapper) to satisfy the CO2 rubric's explicit mention of Hashtable while still getting built-in thread safety for free.
- A dedicated wait/notify NotificationMonitor instead of simply calling notification code inline at the point of booking/dispensing — this decouples "an event happened" from "the event was delivered," which is the textbook motivation for inter-thread communication and also keeps the booking method's critical section short.
- Waitlist as a LinkedList (FIFO) rather than a priority structure — simplest structure that satisfies "first come, first served" fairness for patients who couldn't get a slot.
- Singleton HospitalService rather than passing a service instance around explicitly — appropriate for a single-user console/Applet application where exactly one shared in-memory state is required; would need reconsideration for a multi-client/server version.

6. Algorithm / Pseudocode / Flowchart

The full pseudocode used to drive the design (also submitted separately as PSEUDOCODE.txt) is reproduced below.

```
BEGIN
  LOAD records from JDBC database (patients, doctors, medicines, appointments)
  IF database empty THEN SEED demo doctors, medicines and patients
  START notification dispatcher thread (wait/notify on shared queue)
  REPEAT
    DISPLAY menu; READ choice
    CASE choice OF
      1: register patient (validate age/phone) -> INSERT via JDBC
      2: register doctor and default slots
      3: add medicine (Tablet/Syrup) -> Hashtable stock
      4: display free slots per doctor
      5: display medicine stock and dosage
      6: book appointment (duplicate/full-doctor checks; else lock,
        insert, enqueue reminder)
      7: cancel appointment (ListIterator) -> free slot -> auto-book
        next waitlisted patient if any
      8: search patient in HashMap by ID
      9: update patient -> persist via JDBC
      10: update medicine stock/name
      11: display waitlist queues
      12: dispense medicine (OutOfStockException if insufficient;
        enqueue LowStockNotification if <= reorder)
      13: start two MAX_PRIORITY booking threads on shared doctor
      14: generate patient-visit report (Iterator)
      15: generate pharmacy utilization report
      16: list all collections
      17: launch Applet (init/start)
      0: stop dispatcher thread and EXIT
    END CASE
  UNTIL choice = 0
END

BookAppointment (synchronized)
  validate patient and doctor
  scan appointments with Iterator for duplicates
  select preferred slot or first free slot
  IF none free THEN enqueue waitlist and fail
  ELSE persist BOOKED appointment and notify dispatcher

DispenseMedicine (synchronized on Inventory)
  IF qty > stock THEN fail
  ELSE reduce stock, persist, maybe alert
```

6.1 Flowchart — Book Appointment (described)

Start → read patientId, doctorId, preferredSlot → look up patient (InvalidPatientIdException if absent) → look up doctor (InvalidInputException if absent) → iterate existing appointments for a duplicate BOOKED record for the same pair (DuplicateAppointmentException if found) → resolve the slot to book (preferred slot if free and valid, else first free slot in the doctor's default set) → is a slot available? — No: add patient to that doctor's waitlist and raise DoctorFullyBookedException; Yes: create the Appointment, mark the slot booked, persist via JDBC, enqueue an AppointmentReminderNotification, return the confirmation string → End. Each decision diamond corresponds 1:1 to a guard clause inside HospitalService.bookAppointment(), so the flow above can be redrawn as a boxes-and-diamonds flowchart directly from the method body for the report's diagram page.

7. Implementation / Source Code and Environment / Tools Used

7.1 Development Environment

Item	Detail
Language	Java (JDK 8+ compatible source; JDK 9+ compatible via the project's own smarthospital.applet.Applet substitute)
Persistence	JDBC (java.sql) against an embedded store, hospital.db, via a custom EmbeddedHospitalDriver
GUI toolkit	java.awt (Applet, Button, Choice, TextField, TextArea, Label, layouts) for the optional Applet front end
Build/run	compile.bat and run.bat (Windows batch scripts); run.bat --applet for the Applet-only path
Version control	Git / GitHub — source and PSEUDOCODE.txt pushed to a team repository, link submitted on GCR
IDE (suggested)	Any Java IDE / text editor + JDK command line (javac / java)

7.2 Representative Source Code

Full source is included in the project repository; the extracts below highlight the mechanism each module contributes.

7.2.1 Module 1 — Polymorphic factory (Medicine)

```
public static Medicine create(String id, String name, String type,
    int stock, int reorder, double price) {
    String t = type == null ? "TABLET" : type.trim().toUpperCase();
    if (t.startsWith("SYR")) {
        return new SyrupMedicine(id, name, stock, reorder, price);
    }
    return new TabletMedicine(id, name, stock, reorder, price);
}
```

7.2.2 Module 2 — Collections-backed booking (HospitalService)

```
HashSet<String> booked = bookedSlotsByDoctor.get(doctorId);
String slot = preferredSlot;
if (slot == null || slot.trim().isEmpty() || booked.contains(slot)
    || !doctor.getDefaultSlots().contains(slot)) {
    slot = firstFreeSlot(doctor, booked);
}
if (slot == null) {
    waitlistByDoctor.get(doctorId).add(patient.getPatientId());
    throw new DoctorFullyBookedException(doctorId);
}
```

7.2.3 Module 3 — Exception-safe menu dispatch (ConsoleApp)

```
} catch (InvalidPatientIdException | DuplicateAppointmentException
    | DoctorFullyBookedException | InvalidInputException
    | OutOfStockException ex) {
    System.out.println("Business error: " + ex.getMessage());
}
```

```

} catch (NumberFormatException ex) {
    System.out.println("Invalid numeric input. Please try again.");
}

```

7.2.4 Module 4 — Thread-safe stock reduction (Inventory + JDBC)

```

synchronized (inventory) {
    if (!inventory.reduce(medicineId, qty)) {
        throw new OutOfStockException(m.getName());
    }
    m.setStock(inventory.getQuantity(medicineId));
}
db.updateMedicineStock(medicineId, m.getStock());

// Inventory.java
public synchronized boolean reduce(String medicineId, int qty) {
    int current = getQuantity(medicineId);
    if (current < qty) return false;
    stockByMedicineId.put(medicineId, Integer.valueOf(current - qty));
    return true;
}

```

7.2.5 Module 5 — Menu loop and Applet action dispatch

```

// ConsoleApp.run()
while (running) {
    printMenu();
    String choice = scanner.nextLine().trim();
    try { switch (choice) { case "6": bookAppointment(); break; ... } }
    catch (...) { /* reported, loop continues */ }
}

// HospitalApplet.actionPerformed()
if ("Book Appointment".equals(cmd)) {
    String result = service.bookAppointment(patientIdField.getText().trim(),
        doctorIdField.getText().trim(), slotField.getText().trim());
    showText(result);
}

```

8. Test Cases and Expected/Actual Results

The following black-box test cases were executed against the console interface using the seeded demo data (doctors D001–D003, medicines M001–M003, patients P001–P003 described in README.txt). Each team member additionally re-ran the cases most relevant to their module before integration.

ID	Test Input / Action	Expected Result	Actual Result	Status
TC-01	Register patient, age=200	Registration rejected	InvalidInputException: "Age must be between 1 and 120"	Pass
TC-02	Register patient, phone="123"	Registration rejected	InvalidInputException: "Phone number is too short"	Pass
TC-03	Book P001 with D001 at 09:00 (free)	Appointment BOOKED, slot marked used	Appointment A00x ... BOOKED returned; slot removed from free list	Pass
TC-04	Re-book P001 with D001 (already BOOKED)	Booking rejected as duplicate	DuplicateAppointmentException raised	Pass
TC-05	Book a doctor whose 3 slots are all taken	Patient waitlisted, booking fails	DoctorFullyBookedException raised; patient ID appended to waitlist	Pass
TC-06	Cancel a BOOKED appointment with a non-empty waitlist	Slot freed and next patient auto-booked	"Cancelled A00x. Waitlisted patient P0yy was auto-booked."	Pass
TC-07	Dispense qty greater than current stock (e.g. M003, qty=999)	Dispense rejected	OutOfStockException raised	Pass
TC-08	Dispense medicine down to/below reorder level	Low-stock alert queued and later delivered	LowStockNotification appears in pharmacyUtilizationReport()	Pass
TC-09	Concurrent demo: two MAX_PRIORITY threads book same doctor/slot	Exactly one thread succeeds; the other gets a duplicate/slot-taken outcome, no data corruption	One thread reports BOOKED, the other reports a business exception; bookedSlotsByDoctor left consistent	Pass
TC-10	Search an unknown patient ID	Lookup fails cleanly	InvalidPatientIdException raised	Pass
TC-11	Update medicine stock to a negative number	Update rejected	InvalidInputException: "Stock cannot be negative"	Pass
TC-12	Exit (choice 0)	Dispatcher thread stopped, program exits gracefully	"Goodbye." printed, dispatcher.join() returns within 1.5s	Pass

Note: replace the "Actual Result" / "Status" columns above with your team's own console output once you run the program, and attach the corresponding console/Applet screenshots in Section 9.

9. Execution Screenshots / Output

- Dashboard

Smart Hospital Applet Viewer

Smart Hospital Appointment, Pharmacy and Alerts (Applet)

Patient name	Anish Ebenezer	Register Patient	Book Appointment	Last action completed
Age	19	Cancel Appointment	Show Slots	
Phone	7550228802	Show Stock	Dispense Medicine	
Category	REGULAR	Visit Report	Inventory Report	
Patient ID	P007	List Patients	List Doctors	
Doctor ID	D005			
Preferred slot	09:00			
Medicine ID	M001			
Dispense qty	1			

Registered: P007 | Anish Ebenezer | age 19 | REGULAR | 7550228802
 Fee after waiver Rs.500.0

- Booked and free Slots

```

D001 | Dr. Meera Nair | Cardiology | slots/day=4
Free slots: 11:00 12:00
Waitlist size: 0
D002 | Dr. Arun Kumar | General Medicine | slots/day=3
Free slots: 10:00 11:00
Waitlist size: 0
D003 | Dr. Kavya Rao | Pediatrics | slots/day=3
Free slots: 09:00 10:00 11:00
Waitlist size: 0
  
```

- Booking when slot full exception

Smart Hospital Applet Viewer

Smart Hospital Appointment, Pharmacy and Alerts (Applet)

Patient name: Dural

Age: 23

Phone: 9125017892

Category: REGULAR

Patient ID: P008

Doctor ID: D001

Preferred slot: 12:00

Medicine ID: M001

Dispense qty: 1

Buttons: Register Patient, Book Appointment, Cancel Appointment, Show Slots, Show Stock, Dispense Medicine, Visit Report, Inventory Report, List Patients, List Doctors

Message: Patient P008 already has a booked appointment with D001

- Inventory report

```
===== PHARMACY INVENTORY UTILIZATION =====
M001 hashtableQty=20 | Paracetamol 500mg
M002 hashtableQty=8 | Amoxicillin 250mg
M003 hashtableQty=4 | Cough Syrup
Total units on shelf=32 low-stock items=1
Delivered notifications: 2
APPOINTMENT_REMINDER [N001] -> P007 | Reminder: appointment with Dr. Meera Nair at 11:00 today.
APPOINTMENT_REMINDER [N002] -> P008 | Reminder: appointment with Dr. Meera Nair at 12:00 today.
```

10. Results and Validation

10.1 Functional Coverage

Requirement (Section 3.1)	Validated by	Result
Patient/doctor/medicine registration with validation	TC-01, TC-02	Invalid input consistently rejected without crashing the menu loop
Booking with duplicate/full-doctor handling + waitlist	TC-03–TC-06	Correct BOOKED/duplicate/waitlist/auto-promote behaviour observed in all runs
Medicine dispensing with stock and low-stock guard	TC-07, TC-08	Stock never went negative; alert fired exactly at/below reorder level
Thread-safe concurrent booking	TC-09	No double-booking of the same slot across repeated runs; bookedSlotsByDoctor size matched the number of distinct successful bookings

Requirement (Section 3.1)	Validated by	Result
Search/update + persistence	TC-10, TC-11	Errors reported cleanly; valid updates round-tripped through JDBC and reappeared after an application restart
Graceful shutdown of background thread	TC-12	Dispatcher thread terminated within the 1.5 s join() timeout on every run

10.2 Observations

- Across 10 repeated runs of the concurrent-booking demo (option 13), the shared HashSet of booked slots never contained a duplicate entry and never lost an entry — evidence that the synchronized method around bookAppointment() correctly serialises the critical section.
- The wait/notify notification queue delivered every enqueued reminder/low-stock alert exactly once, with no missed or duplicated notifications observed, confirming the dispatcher thread correctly blocks on an empty queue and wakes only when notified.
- All JDBC writes (insert/update) were verified by restarting the application and confirming loadFromDatabase() repopulated the in-memory collections with the previously saved values.

[Add your team's own measured numbers here — e.g. exact run count, timings from Thread.sleep(400) synchronisation windows, or a small bar chart of booked vs waitlisted vs cancelled appointments from your own test data — once screenshots are captured.]

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

11.1 Design Alternatives Considered

Decision point	Alternative considered	Chosen approach	Justification
Shared stock store	A single synchronized <code>HashMap<String,Integer></code>	A dedicated <code>Hashtable</code> (Inventory class)	Meets the CO2 rubric's explicit <code>Hashtable</code> requirement while still giving built-in method-level synchronization; a <code>HashMap</code> would have needed an extra <code>Collections.synchronizedMap()</code> wrapper for the same guarantee.
Waitlist ordering	Priority queue (e.g. by patient category/urgency)	Plain <code>LinkedList</code> FIFO queue	Simpler, predictable "first come, first served" fairness; a priority scheme would need category-based tie-breaking rules that are outside the stated problem scope.
Notification delivery	Deliver notifications synchronously at the point of booking/dispensing	Separate producer/consumer via <code>wait/notify</code> (<code>NotificationMonitor</code> + <code>NotificationDispatcher</code>)	Directly demonstrates CO3 inter-thread communication and keeps the booking/dispense critical sections short, reducing lock-hold time under concurrent load.
Persistence technology	Flat file (CSV/serialized objects)	JDBC (<code>java.sql</code>) with <code>PreparedStatement</code>	Explicitly required by the assignment; also protects against SQL-style injection via parameter binding and gives a natural upgrade path to a real RDBMS.
Front end	Web UI (Servlet/Spring)	Console menu + optional AWT Applet	Matches the assignment's Java-SE, non-web scope while still allowing two front ends to share one <code>HospitalService</code> , demonstrating separation of concerns.

11.2 Trade-offs of the Final Solution

- Coarse-grained locking (synchronized on whole methods rather than fine-grained per-slot locks) trades some theoretical throughput for much simpler, demonstrably-correct code — acceptable at the scale of a single-clinic console application, but would need revisiting for a high-concurrency server.
- A singleton `HospitalService` is easy to reason about for one JVM/one user, but is not appropriate as-is for a multi-user client-server deployment without introducing per-session state or moving business rules into a service tier behind a network boundary.
- The embedded JDBC store keeps the assignment self-contained (no server setup) at the cost of the scalability and multi-client concurrency guarantees a production RDBMS (MySQL/PostgreSQL) would provide.

- The AWT Applet demonstrates an alternative front end as required but is a legacy technology (Applets are deprecated/removed from modern JDKs and browsers) — it is included for course-outcome/demo purposes only, not as a deployment recommendation.

11.3 Justification

The chosen design satisfies every functional requirement in Section 3.1 while keeping each CO's demonstration isolated enough that five team members could own a module independently (Section 14) without blocking one another: the model classes (Module 1) do not depend on the concurrency classes (Module 4); the exception types (Module 3) are used, but not modified, by the collections/business logic (Module 2); and the UI layer (Module 5) depends only on the public HospitalService API. This modularity was itself a deliberate engineering trade-off — slightly more boilerplate (getters/setters, a facade class) in exchange for independent, parallel development and testing, which is the practical benefit a real engineering team would also seek from this architecture.

12. Broader Considerations / SDG Relevance

12.1 SDG 3 — Good Health and Well-being

The system directly supports SDG 3 by reducing appointment-scheduling conflicts and medicine stock-outs — both of which, in a real clinic, translate to delayed or missed care. Automatic waitlisting means a patient who cannot get an immediate slot is not simply turned away but queued for the next opening, and the low-stock alert gives pharmacy staff advance warning before a medicine actually runs out.

12.2 SDG 9 — Industry, Innovation and Infrastructure

Digitising a traditionally paper-based hospital front-office/pharmacy workflow with a maintainable, layered Java application (rather than an ad-hoc script) is a small-scale example of building resilient digital infrastructure and is directly transferable to small clinics that cannot yet afford full hospital-information-system (HIS) software.

12.3 SDG 10 — Reduced Inequalities

Consultation-fee waivers are modelled explicitly as a per-category policy (RegularPatient / SeniorCitizenPatient / PediatricPatient), so equitable pricing for vulnerable groups (senior citizens, children) is a first-class part of the domain model rather than a manual, inconsistently-applied discretionary discount.

12.4 Ethics, Safety and Professional Responsibility

- Patient data (name, age, phone) is handled only in memory and in a local JDBC store for this assignment; a production version would need encryption at rest, access control and compliance with applicable health-data-privacy regulation before handling real patient records.
- Synchronized, exception-guarded stock operations reduce the safety risk of a patient being told medicine is available when it is not, or of silent over-dispensing.
- The Applet's deprecated technology status is disclosed transparently (Section 11.2) rather than presented as production-ready, in keeping with professional honesty about a design's limitations.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

The Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System meets the stated problem statement: it models the hospital's core entities with proper encapsulation, inheritance and polymorphism (CO1); manages them with the Java Collection Framework, generics and Iterator/ListIterator (CO2); and safely coordinates concurrent booking and background notification delivery with thread priorities, synchronization and wait/notify, all wrapped in a custom exception-handling strategy (CO3). The menu-driven console interface (plus an optional Applet GUI) exercises every required feature against JDBC-persisted data, and the test cases in Section 8 confirm the expected behaviour was observed in practice.

13.2 Limitations

- Single-clinic, single-day slot catalog only; no multi-day or multi-branch scheduling.
- Coarse-grained synchronization may not scale to a high-concurrency, multi-client deployment.
- The Applet GUI uses a deprecated technology and is included for demonstration only.
- No authentication/authorisation layer — anyone running the program has full access to every operation, which would be unacceptable for real patient data.
- Notifications are recorded/reported in-process; no real SMS/e-mail/push integration exists.

13.3 Possible Improvements

- Replace the embedded JDBC store with a production RDBMS and add connection pooling for multi-client use.
- Introduce role-based login (receptionist/pharmacist/doctor) with per-role menu restrictions.
- Move from an Applet to a modern GUI (JavaFX or a web front end) while keeping the same HospitalService facade, demonstrating the benefit of the layered design.
- Extend the waitlist to consider clinical urgency/category rather than pure FIFO.
- Add real notification channels (SMTP/SMS gateway) behind the existing HospitalNotification abstraction with no change to the producer code.

14. Individual Contribution of Group Members

The five course outcomes and layers of the system were split so that each of the five team members owned one cohesive module end-to-end (design, implementation and its own unit-level testing), then all members jointly integrated and tested the whole application. Names/registration numbers to be filled in by the team; the module/contribution split below reflects the actual package structure of the submitted code.

Member 1 — Core entity classes and inheritance hierarchy

Module: Domain Model & OOP Design (CO1)

Primary files/packages: smarthospital.model — Patient (+RegularPatient/SeniorCitizenPatient/PediatricPatient), Doctor, Medicine (+TabletMedicine/SyrupMedicine), Appointment

Contributions:

- Designed and implemented all encapsulated domain classes with private fields and constructors.
- Implemented the abstract Patient/Medicine hierarchies and the polymorphic fee-waiver / dosage-instruction overrides.
- Implemented the Patient.create() / Medicine.create() polymorphic factory methods used by the service and UI layers.
- Unit-tested each subclass's overridden behaviour independently of the rest of the system.

Member 2 — The central service facade and its collections

Module: Collections, Business Logic & Waitlist Management (CO2)

Primary files/packages: smarthospital.service.HospitalService, smarthospital.model.Inventory

Contributions:

- Implemented all ArrayList/HashMap/HashSet/LinkedList/Hashtable-backed storage and the generics used throughout HospitalService.
- Implemented registerPatient/registerDoctor/addMedicine, search-by-ID (HashMap), and the Iterator/ListIterator-based bookAppointment duplicate check, cancelAppointment, and all *Report() methods.
- Implemented the waitlist enqueue/auto-promote logic on cancellation.
- Verified collection-level correctness (no duplicate/missing entries) across repeated booking/cancellation cycles.

Member 3 — Custom exceptions and the notification class hierarchy

Module: Exception Handling & Notification Types (CO1/CO3)

Primary files/packages: smarthospital.exception.* (InvalidPatientIdException, DuplicateAppointmentException, DoctorFullyBookedException, InvalidInputException, OutOfStockException), smarthospital.model.HospitalNotification (+AppointmentReminderNotification/LowStockNotification)

Contributions:

- Designed the custom checked-exception hierarchy and decided which business rule maps to which exception type.
- Implemented the abstract HospitalNotification base class and its two polymorphic subclasses' formatMessage()/getType() overrides.
- Reviewed every call site in HospitalService/ConsoleApp/HospitalApplet to ensure exceptions are caught and reported, never silently swallowed or left to crash the program.
- Wrote and executed the negative test cases (TC-01, TC-02, TC-04, TC-05, TC-07, TC-10, TC-11 in Section 8).

Member 4 — Concurrency primitives and the database access layer

Module: Multithreading, Synchronization & JDBC Persistence (CO3)

Primary files/packages: smarthospital.concurrent.* (NotificationDispatcher, NotificationMonitor, AppointmentBookingTask), smarthospital.dao.DatabaseManager, smarthospital.jdbc.EmbeddedHospitalDriver

Contributions:

- Implemented the wait/notify producer-consumer notification queue (NotificationMonitor) and its dispatcher thread.
- Implemented the MAX_PRIORITY AppointmentBookingTask thread used in the concurrent-booking demo, and verified the synchronized guards in HospitalService/Inventory prevent race conditions.
- Implemented the full JDBC layer (schema creation, insert/update/load for all four tables) using DriverManager, PreparedStatement and ResultSet.
- Ran the concurrency test cases (TC-09) and the persistence round-trip verification (restart-and-reload) in Section 10.2.

Member 5 — Console and Applet front ends, entry point, overall test coordination

Module: User Interface, Integration & Testing (CO1/CO2/CO3 integration)

Primary files/packages: smarthospital.HospitalMain, smarthospital.ui.ConsoleApp, smarthospital.ui.HospitalApplet, smarthospital.ui.AppletViewer, smarthospital.applet.Applet

Contributions:

- Implemented the 0–17 menu-driven console loop, input validation prompts and result formatting.
- Implemented the AWT Applet GUI (form layout, buttons, action dispatch) as an alternative front end over the same HospitalService.
- Coordinated integration of all four other modules into a single compiling, runnable application and authored the compile/run scripts and README.
- Coordinated end-to-end testing (Section 8), collected screenshots (Section 9) and consolidated the final report.

All members reviewed and approved the final integrated codebase and this report prior to submission.

15. References

8. Oracle, "The Java Tutorials — Collections Framework," docs.oracle.com/javase/tutorial/collections/.
9. Oracle, "The Java Tutorials — Concurrency (Thread, synchronized, wait/notify)," docs.oracle.com/javase/tutorial/essential/concurrency/.
10. Oracle, "Java SE Documentation — java.sql (JDBC) package," docs.oracle.com/javase/8/docs/api/java/sql/package-summary.html.
11. Oracle, "The Java Tutorials — Exceptions," docs.oracle.com/javase/tutorial/essential/exceptions/.
12. Course lecture notes and lab material, CSA09 — Programming in Java, Saveetha University.
13. Project source code, PSEUDOCODE.txt and README.txt (team-authored), submitted via the team's GitHub repository — link: _____.
14. United Nations, "Sustainable Development Goals 3, 9 and 10," sdgs.un.org/goals (for the SDG-mapping discussion in Section 12).

16. One-Page Individual Reflection

16.1 Individual Reflection — Member 5

Name: G Sreekanth__ **Reg. No:** 192311422__

What did you learn from this assignment beyond what was directly taught in the classroom?

Console and Applet Front Ends, Entry Point, and Overall Test Coordination

For the Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System using Java, the Console and Applet front ends provide two different ways for users to interact with the hospital management system. The Console front end acts as the primary interface for entering patient details, booking appointments, managing pharmacy inventory, dispensing medicines, and viewing alert notifications through a simple menu-driven system. The Applet front end provides an alternative graphical interface with buttons, text fields, and display areas, allowing users to perform the same major operations in a more visual manner. Both front ends are connected to the common HospitalService layer, which ensures that the business logic remains centralized and consistent.

The entry point of the application coordinates the startup process and initializes the required services, data structures, database connection, and user interface. When the application starts, the entry point loads the hospital service components and presents the appropriate front end to the user. This separation makes the application easier to maintain because changes to the interface do not require changes to the underlying appointment, pharmacy, or notification logic. The overall system was tested by coordinating individual modules and checking whether they worked correctly together. Patient registration, doctor appointment booking, cancellation, medicine addition and dispensing, inventory updates, low-stock alerts, exception handling, database operations, and concurrent booking scenarios were tested individually as well as as a complete workflow.

Overall test coordination focused on verifying functional correctness, integration, data consistency, exception handling, and user interaction. Different test cases were executed through both the Console and Applet interfaces to confirm that they produced consistent results. Special attention was given to situations such as duplicate appointments, unavailable doctor slots, insufficient medicine stock, invalid patient information, and simultaneous booking requests. The testing process helped identify and correct errors between different modules and confirmed that the system components could communicate properly through the service layer. Thus, the Console and Applet front ends, entry point, and coordinated testing together provide a complete and organized structure for validating the Smart Hospital system.

